Groupe	Membre 1	Membre 2

Projet Push Swap - Liste des groupe

Membre 3	Note

es

Remarque

Nom prenom Critères
C1 – Gestion des arguments et initialisation des piles
C2 – Implémentation des instructions Push-Swap
C3 – Programme push-swap : tri fonctionnel
C4 – Optimisation du nombre d’opérations
C5 – Programme checker
C6 – Gestion des erreurs
C7 – Respect des contraintes du sujet
C8 – Algorithmique et manipulation des piles
C9 – Architecture et organisation du projet

C10 – Qualité et lisibilité du code
C11 – Robustesse et cas limites
Bonus

Push Swap - Grille d'évaluati

Détails	Barème
Fonctionnalités obligatoires	**12**
* Le programme accepte une liste d'entiers en argument et initialise correctement la pile a. * Le premier entier reçu correspond au sommet de la pile. * La pile b est correctement initialisée. * En l'absence d'argument, le programme ne produit aucune sortie.	1,5
* Les instructions pa, pb, sa, sb, ss, ra, rb, rr, rra, rrb et rrr sont correctement implémentées. * Chaque instruction modifie les piles conformément au sujet. * Les opérations combinées (ss, rr, rrr) produisent le comportement attendu.	2,5
* L'exécutable push-swap trie correctement la pile a dans l'ordre croissant. * La pile b est utilisée lorsque nécessaire. * La sortie contient uniquement les instructions nécessaires, séparées par \n. * La séquence générée permet d'obtenir une pile a triée et une pile b vide.	3
* Le programme cherche à produire une séquence d'instructions courte. * Les choix d'algorithme évitent les opérations manifestement inutiles ou redondantes. * La solution reste cohérente avec l'objectif du sujet : trier avec le plus petit nombre d'opérations possible.	1,5
* L'exécutable checker lit les instructions depuis l'entrée standard. * Les instructions sont exécutées dans le bon ordre sur les piles. * Le programme affiche OK si a est triée et b vide. * Dans les autres cas, il affiche KO. * Sans argument, il ne produit aucune sortie.	2
* Les arguments non entiers sont détectés. * Les doublons sont détectés. * Les instructions inconnues ou mal formatées sont rejetées par checker. * En cas d'erreur, Error\n est écrit sur la sortie d'erreur. * Le programme ne plante pas sur les cas d'erreur attendus.	1,5
Interface et parcours utilisateur	8
* Le projet est écrit en Go. * Seuls les packages de la bibliothèque standard Go sont utilisés. * Les exécutables attendus sont nommés checker et push-swap. * Le projet peut être compilé et exécuté conformément aux exemples d'utilisation fournis.	1,5
* Les piles sont représentées avec des structures de données cohérentes. * Les opérations de pile sont centralisées et réutilisables. * L'algorithme de tri est compréhensible et adapté au problème. * La logique de génération des instructions est clairement identifiable.	2
* L'arborescence du projet est claire. * Les responsabilités sont séparées : parsing, validation, opérations, tri, checker, affichage. * Les fichiers, fonctions et packages sont nommés de manière cohérente. * Le code peut être compris rapidement par une personne extérieure.	1,5

* Le code respecte les bonnes pratiques Go attendues. * Les fonctions restent ciblées et évitent les responsabilités multiples. * La duplication de code est limitée. * Les conditions et boucles restent lisibles. * Les commentaires sont utilisés lorsqu'ils apportent une réelle valeur.	1,5
* Les cas limites principaux sont pris en compte : liste déjà triée, une seule valeur, plusieurs valeurs, entrée invalide, doublons. * Les opérations sur des piles vides ou insuffisamment remplies ne provoquent pas de comportement incohérent. * L'exécution reste stable sur différents jeux de données valides.	1,5
	1
	1

on projet

État d'acquisition
Non acquis
Non acquis
Non acquis
Non acquis
Non acquis
Non acquis
Non acquis
Non acquis
Non acquis

Non acquis
Non acquis
Non acquis
Moyenne (sur 20)

Remarque	Points
	0
	0
	0
	0
	0
	0
	0
	0
	0
	0
	0

	0
	0
	0
	0
0	

Label	Pondération
Non acquis	0
En cours d’acquisition	0,35
Partiellement acquis	0,55
Acquis	0,75
Maîtrisé	1